```
  def this_doesnt_end
    return "a thing"

  def this_one_is_also_right
    p "fine"
  end
end
```

Ruby will do this:

```
sh: code/rubyworld/syntax_error.rb: No such file or directory
```

Ruby notices the error—a missing end—and attempts to locate the actual location of the item missing the end. In this case, it gets it right.

Any options after the command ruby are sent to the Ruby interpreter. The Ruby options end with the first word on the command line that doesn't start with a hyphen or by the special flag -- (two hyphens).

There are ways to invoke the Ruby interpreter without passing it a filename. One way is to use the -e command-line option, which executes one line of script.

This lets us use Ruby as a powerful command-line calculator. Here's a one liner that returns the first five square numbers:

```
ruby -e "p (1..5).map { _1 ** 2 }"
[1, 4, 9, 16, 25]
```

When you do this, remember that the command you run needs to be a string, and that you have to print it or you won't see the result.

You can also pipe a file into the command using Unix standard input and then access that file using Kernel#gets:

```
ruby -e 'puts "line: #{gets}"' < testfile
```

That works swimmingly, but it does only process the first line of the file. However, we can use Ruby's while expression clause to loop over the file in a single line:

```
$ ruby -e 'puts "line: #{$_}" while gets' < testfile
line: This is line one
line: This is line two
line: This is line three
line: And so on...
```

In this snippet, we're not only taking advantage of the while clause, we're also using a Ruby global, $_, which contains the most recent text read in by a gets call. So, the while gets reads the line and puts it in $_ and the body of the statement prints out the line.

Still, that while at the end seems kind of awkward for something you might do often. If only there were some kind of shortcut:

```
$ ruby -ne 'puts "line: #{$_}"' < testfile
line: This is line one
line: This is line two
line: This is line three
line: And so on...
```